[NLP] BERT : Pre-training of Deep Bidirectional Transformers for Language Understanding

교재 내용 정리

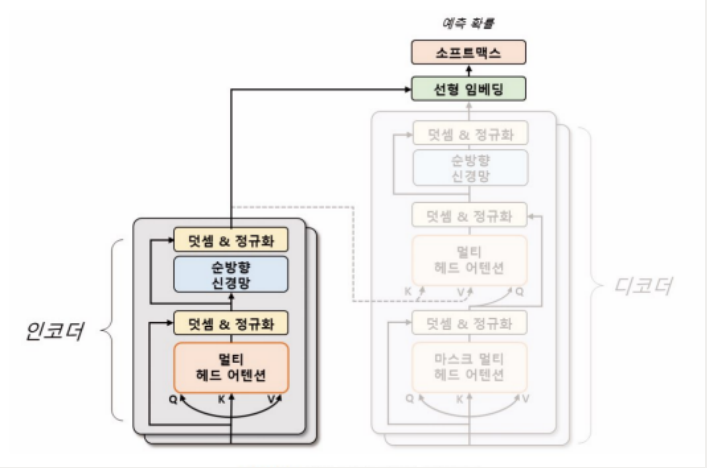
BERT

📖 **BERT (Bidirectional Encoder Representations from Transformers)**

> 구글에서 발표한 언어 모델로 트랜스포머 기반 양방향 인코더를 사용하는 자연어 처리 모델

🔹 양방향 (Bidirectional) 인코더
: 양방향으로 문장을 학습하므로 이전과 이후의 단어를 모두 참조해 단어의 의미와 문맥 파악 가능
➡️ 기존 모델들보다 더 정확하게 문맥을 파악하고, 다양한 자연어 처리 작업에서 높은 성능을 보임.

➕ BERT는 대규모 데이터를 사용해 사전 학습되어 있으므로 전이 학습에 주로 활용됨.



🔹 BERT의 트랜스포머 인코더 모듈
➡️ 입력 문장의 단어를 좌우 양방향으로 처리해 문맥 정보를 모델링 하므로 인코더 계층만을 사용해 학습

사전 학습 방법

📖 **사전 학습을 위해 마스킹된 언어 모델링(MLM)과 다음 문장 예측 방법(NSP)을 사용함.**

1️⃣ 마스킹된 언어 모델링 (Masked Language Modeling, MLM)

> 입력 문장에서 임의로 일부 단어를 마스킹하고 해당 단어를 예측하는 방식

➡️ 입력 문장에서 누락된 단어를 추론
✅ 문장 전체 의미를 이해하는 능력 향상

2️⃣ 다음 문장 예측 (Next Sentence Prediction, NSP)

> 두 개의 문장이 주어졌을 때, 두 번째 문장이 첫 번째 문장의 다음에 오는 문장인지 여부를 판단하는 작업

➡️ 두 문장이 연속적인 관계인지 아닌지를 예측
✅ 두 문장 간의 관계를 학습하고, 문장 간의 의미적인 유사성을 파악

3️⃣ 특수 토큰
➡️ 입력 문장에 특수한 토큰들을 추가해 모델이 학습하고 추론하는 과정에서 필요한 정보를 제공

🔹 [CLS] 토큰

> 입력 문장의 시작 부분에 추가되는 토큰

➡️ 문장 분류 작업을 위한 정보 제공
✅ 입력 문장이 어떤 유형이 문장인지 미리 파악 가능

🔹 [SEP] 토큰

> 입력 문장 내에서 두 개 이상의 문장을 구분하기 위해 사용되는 토큰

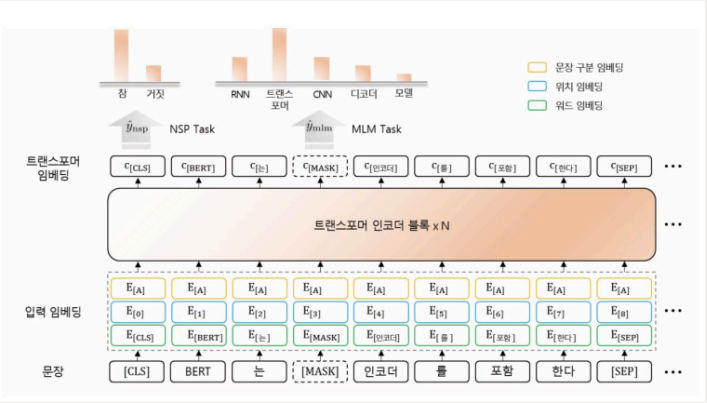
➡️ 입력 문장을 두 개의 독립적인 문장으로 인식
✅ 각각의 문장에 대한 정보를 정확하게 파악 가능

🔹 [MASK] 토큰

> 입력 문장 내에서 임의로 선택된 단어를 가리키는 토큰

➡️ 주어진 문장에서 일부 단어를 가린 후 모델의 학습과 예측에 활용

🔹 BERT 모델의 입력 임베딩과 손실 함수



➡️ [CLS] BERT는 트랜스포머 인코더를 포함한다 [SEP] 와 같은 형태로 변환
➡️ [MASK] 토큰 적용시
텍스트 토큰 중 15%에 해당하는 단어를 대상을 마스킹 수행

```
80% → [MASK] 토큰 대체
10% → 어휘 사전에 존재하는
          무작위 단어로 변경
10% → 실제 토큰 그대로 사용
```

➕ 사전 학습 수행 후, 미세 조정 기법을 통해 다양한 자연어 처리 작업(문장 분류, 감성 분석, 질문 응답, 기계번역 등)에 적용 가능
➕ 미세 조정 과정에서는 해당 작업에 맞는 계층을 추가하고 손실 함수 정의해 학습 가능

0. Abstract

📢 **BERT라는 새로운 언어 표현 모델을 제안**
→ Transformer 기반의 양방향(Bidirectional) 표현 모델
✅ 양방향 (앞 + 뒤 동시에 확인)
➡️ 단어를 이해할 때 앞 뒤 문맥을 동시에 고려
⭕ 별도의 복잡한 구조 변경 없이 사용 가능 + 다양한 task에 fine-tuning 가능

1. Introduction

논문이 다루는 분야

📌 **자연어처리 (NLP, Natural Language Processing)**
👉 Language Representation Learning (언어 표현 학습)

> 대규모 텍스트로부터 일반적인 언어 표현을 사전학습(pre-training)하고 이를 다양한 NLP task (QA, NLI, classification 등)에 활용

해당 task에서 기존 연구 한계점

🚨 **기존 접근 방식**

1️⃣ **Feature-based 방식 (ELMo)**

- 사전 학습된 언어 표현을 입력 feature로 사용
- 각 task에 맞는 모델을 따로 설계해야 함

✅ 양방향 정보를 사용하긴 하나 좌→우 / 우→좌를 따로 학습 한 후 결합
❌ 완전한 양방향 문맥 학습이 아님
❌ task마다 구조 설계 필요 → 비효율적

2️⃣ **Fine-tunning 방식 (GPT)**

- 사전 학습된 모델을 그대로 가져와 전체를 task에 맞게 학습
- 하나의 모델을 다양한 task에 적용 가능

✅ 구조 단순 + 범용성 높음
❌ 문장의 앞부분만 보고 다음 단어를 예측하는 방식으로 학습 (단방향)
❌ 전체 문맥을 동시에 반영하지 못함.

> **완전한 양방향 문맥 학습 불가능**

→ 문장을 부분적으로만 이해하고 문장 관계가 중요한 task에서 성능 제한

논문의 contributions

💡 1️⃣ **Deep Bidirectional Language Model 제안**
Transformer Encoder 기반으로
➡️ 문장의 앞뒤 문맥을 동시에 고려하는 양방향 표현 학습 가능
✅ 기존 GPT (단방향) / ELMo (분리된 양방향) 한계 해결

2️⃣ **Masked Language Model (MLM)**
➡️ 입력 문장의 일부 단어를 가리고 이를 예측하도록 학습
✅ 단어를 이해할 때 앞 + 뒤 문맥을 모두 활용 가능
✅ 양방향 학습 가능하게 만든 핵심 방법

3️⃣ **Next Sentence Prediction (NSP)**
➡️ 두 문장이 이어지는 관계인지 예측하는 task 추가
✅ 문장 간 관계 학습
✅ QA, NLI 등 문장 단위 task 성능 향상

4️⃣ **Pretrain + Fine-tune 구조 확립**
➡️ 하나의 모델을 사전 학습 후 다양한 task에 적용
✅ task별 모델 설계 불 필요

5️⃣ **다양한 NLP task 에서 SOTA 달성**
➡️ GLUE, SQuAD 등 주요 benchmark에서 최고 성능

📢 이 논문에서는 **BERT라는 양방향 문맥을 학습하는 새로운 pre-training 방식으로 NLP의 표준 구조를 만든 모델**을 제안한다.

2. Background

📖 1️⃣ **Feature-based 방식 (ELMo)**

- Word2Vec, GloVe → 단어 임베딩 학습
- ELMo → 문맥 기반 단어 표현

➡️ 좌→우 / 우→좌 각각 학습 후 결합
❎ 완전한 양방향 아님
❎ task마다 모델 따로 필요

2️⃣ **Fine-tunning 방식 (GPT)**

- 사전 모델을 그대로 task에 맞게 학습

✅ 구조 단순, 다양한 task에 적용 가능
❎ 문장의 앞부분만 사용 (단방향)
❎ 전체 문맥 반영 불가능

🚨 **핵심 문제**

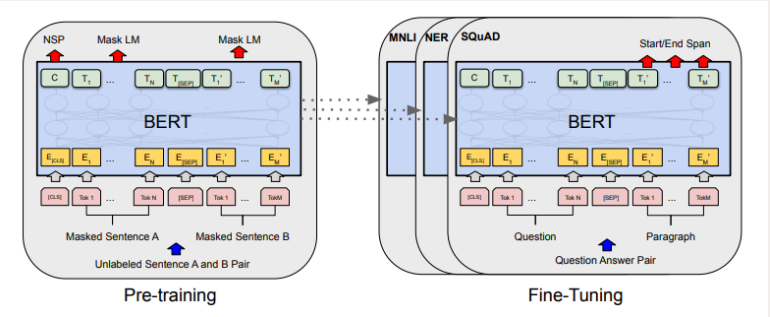
- ELMo → 양방향이지만 분리된 구조
- GPT → 단방향 구조

➡️ 공통적으로 전체 문맥을 동시에 활용 못 함.

3. 제안 방법론

BERT + Input / Output Representation

💡 BERT 구조 핵심 = Pre-training → Fine-tunning



👉 대규모 텍스트 데이터를 이용해 사전학습을 수행하고, 이후 특정 task에 맞게 fine-tunning을 진행

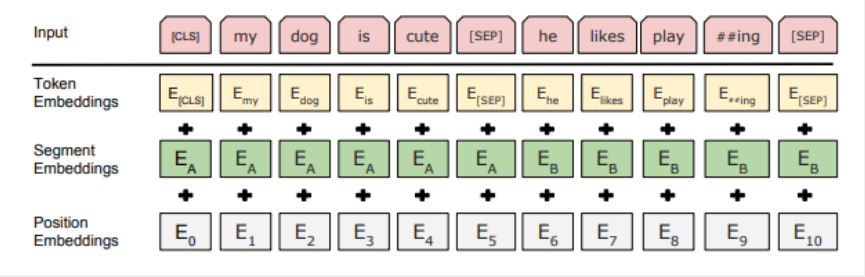
1️⃣ Pre-training

: 라벨 없는 텍스트 데이터를 사용해 모델 학습

2️⃣ Fine-tunning

: 라벨이 있는 데이터를 사용하여 특정 task에 맞게 모델 학습 (예 : QA, 감정 분석 같은 실제 문제)

👉 이 과정에서 모델의 모든 파라미터가 함께 업데이트 = 일부만 바꾸는 게 아니라 전체 모델 다 학습



3️⃣ Input Representation

🔹 입력 구조

- 하나의 시퀀스로 구성
  → 단일 문장 또는 문장 쌍 모두 표현 가능
- 문장 쌍 입력 형태

4️⃣ Output Representation

▼ BERT 출력
BERT에 문장을 넣으면

- [CLS] → 전체 문장 대표 벡터

→ [CLS] + 문장 A + [SEP] + 문장 B

🔹 토큰 단위 구성

각 토큰의 입력을 다음 3가지 임베딩의 합으로 구성

- Token Embedding
  → 단어 자체 의미
- Segement Embedding
  → 문장 A / B 구분
- Position Embedding
  → 단어 순서 정보

🔹 특수 토큰

- [CLS]

➡️ 전체 시퀀스를 대표하는 벡터
➡️ 분류 task에서 사용

- [SEP]

➡️ 문장 구분 역할

🔹 기타 특징

- WordPiece (30,000 vocab) 사용
- "sentence"는 반드시 문법적 문장이 아닌 텍스트 구간 의미
- 각 단어 → 각각 벡터 하나씩 나옴

"나는 학교에 갔다"
결과) T1 = "나는"
T2 = "학교에"
T3 = "갔다"

🔹 [CLS] 벡터

- [CLS] 토큰의 최종 hidden state = $\mathbf{C} \in \mathbb{R}^H$
- 전체 시퀀스를 대표하는 벡터

👉 문장 분류, 문장 관계 task에 사용

🔹 토큰별 출력

- 각 토큰의 최종 hidden state = $\mathbf{T}_i \in \mathbb{R}^H$
  → 각 단어에 대해 각각 벡터 하나씩 나옴

👉 NER, QA(단어 단위 문제)에 사용

🔹 핵심 결과

> 하나의 BERT 출력으로 다양한 task 처리 가능

Pre-training BERT

💡 **BERT는 양방향으로 문맥을 고려하는 사전 학습 수행**

⚙️ **기본 구조**

BERT는 기존 단 방향 언어모델(GPT, ELMo)와 달리
→ 양방향 문맥을 동시에 고려하는 방식으로 사전 학습 수행
⇒ 모든 layer에서 left + right context를 함께 사용하여 representation 학습

1️⃣ Masked Language Model (MLM)

> 입력 문장의 일부 토큰을 가리고 가려진 단어를 예측하는 방식의 언어 학습 모델

🔹 입력 형태
기본 입력 시퀀스 : [CLS] + 문장 + [SEP]
➡️ 일부 토큰을 선택하여 masking 적용

🔹 학습 방식
: 선택된 토큰의 위치에서 원래 단어를 맞추도록 학습

- 출력
  → vocabulary 전체에 대한 확률 분포

➡️ loss : masked 위치에서만 계산

🔹 Masking 전략
: 전체 토큰 중 15% 선택

  - 80% : [MASK]로 변경
  - 10% : random token으로 변경
  - 10% : 그대로 유지

✅ 단어를 이해할 때 앞 뒤 문맥을 동시에 활용 가능
👉 Bidirectional learning 가능
✅ 기존 단방향 language model 한계 해결

2️⃣ Next Sentence Prediction (NSP)

> 두 문장 A, B가 주어졌을 때 B가 A의 다음 문장인지 여부를 이진 분류하는 task

🔹 입력 형태
하나의 시퀀스로 구성
: [CLS] + 문장 A + [SEP] + 문장 B
➡️ [CLS]의 최종 hidden state를 사용하여 문장 관계 판단

🔹 데이터 구성
: 전체 데이터 중

- 50% → 실제 이어지는 문장 (IsNext)
- 50% → 랜덤 문장 (NotNext)

🔹 학습 방식
: [CLS] 벡터 → classification layer 통과

- 출력
  → IsNext / NotNext 확률

➡️ loss : binary classification loss

✅ 단순한 언어 모델이 아니라 문장 간 관계를 이해하려는 능력 학습
👉 QA, NLI와 같은 task에서 중요한 문장 관계 이해 학습 진행

➕ Pre-training 데이터

- 사전 학습 : BooksCorpus(800M words)와 Wikipedia(2500M workds)를 사용

Fine-tunning

💡 **사전학습된 BERT를 기반으로 다양한 NLP task에 맞게 추가 학습 수행**

⚙️ **기본 구조**

Pre-trained BERT에 task-specific layer만 추가
⇒ 전체 파라미터를 함께 학습 (end-to-end)

1️⃣ 입력 구성

> task에 따라 입력 형태만 다르게 구성

🔹 단일 문장
: [CLS] + 문장 + [SEP]
🔹 문장 쌍
: [CLS] + 문장 A + [SEP] + 문장 B
➡️ Pre-training과 동일한 입력 구조 유지

2️⃣ 출력 활용 방식

> BERT의 출력 벡터를 task에 맞게 활용

🔹 [CLS] 벡터 (C)

- 전체 시퀀스를 대표하는 벡터
- 문장 단위 task에 사용

👉 문장 분류, 문장 관계(NLI 등) 사용

🔹 토큰별 벡터 (T_i)

- 각 단어에 대응되는 벡터

👉 NER (개체명 인식), QA (정답 시작 / 끝 위치 예측) 사용

3️⃣ 학습 방식

> 간단한 출력 layer 추가 후 전체 모델 학습

🔹 구조

- [CLS] → classifier → label
- T_i → token-level classifier

➡️ loss는 task에 따라 다르게 설정

4️⃣ **특징**

- 모델 구조 거의 변경 없음

➡️ 기존 BERT 그대로 사용

- 다양한 task에 동일 모델 적용 가능

➡️ 입력만 변경하면 됨

5️⃣ **Self-attention 기반 장점**

- 모든 토큰이 서로를 참고 가능

➡️ 문장 / 문장 쌍 모두 자연스럽게 처리

- 문장 간 관계도 별도 구조 없이 학습 가능

➕ Pre-training과의 연결

- MLM → 단어 수준 의미 학습
- NSP → 문장 관계 학습

👉 Fine-tuning에서 바로 활용 가능

4. 실험 및 결과

1️⃣ GLUE Benchmark

System	MNLI-(m/mm) 392k	QQP 363k	QNLI 108k	SST-2 67k	CoLA 8.5k	STS-B 5.7k	MRPC 3.5k	RTE 2.5k	**Average** -
Pre-OpenAI SOTA	80.6/80.1	66.1	82.3	93.2	35.0	81.0	86.0	61.7	74.0
BiLSTM+ELMo+Attn	76.4/76.1	64.8	79.8	90.4	36.0	73.3	84.9	56.8	71.0
OpenAI GPT	82.1/81.4	70.3	87.4	91.3	45.4	80.0	82.3	56.0	75.1
$BERT_{BASE}$	84.6/83.4	71.2	90.5	93.5	52.1	85.8	88.9	66.4	79.6
$BERT_{LARGE}$	**86.7/85.9**	**72.1**	**92.7**	**94.9**	**60.5**	**86.5**	**89.3**	**70.1**	**82.1**

> 다양한 자연어 이해 task를 포함한 종합 benchmark

🔹 평가 방식

- 입력: 단일 문장 또는 문장 쌍
- 출력: [CLS] 벡터 → classification

➡️ loss: cross-entropy

🔹 결과

- BERTBASE, BERTLARGE 모두
  → 기존 모델 대비 높은 성능 달성
- 특히 BERTLARGE
  → 대부분 task에서 SOTA 달성

🔹 특징

- 단일 모델로 다양한 task 처리 가능
- 기존 task-specific 모델보다 성능 우수

2️⃣ SQuAD v1.1 (Question Answering)

System	Dev		Test	
	EM	F1	EM	F1
Top Leaderboard Systems (Dec 10th, 2018)				
Human	-	-	82.3	91.2
#1 Ensemble - nlnet	-	-	86.0	91.7
#2 Ensemble - QANet	-	-	84.5	90.5
Published				
BiDAF+ELMo (Single)	-	85.6	-	85.8
R.M. Reader (Ensemble)	81.2	87.9	82.3	88.5
Ours				
$\text{BERT}_{\text{BASE}}$ (Single)	80.8	88.5	-	-
$\text{BERT}_{\text{LARGE}}$ (Single)	84.1	90.9	-	-
$\text{BERT}_{\text{LARGE}}$ (Ensemble)	85.8	91.8	-	-
$\text{BERT}_{\text{LARGE}}$ (Sgl.+TriviaQA)	**84.2**	**91.1**	**85.1**	**91.8**
$\text{BERT}_{\text{LARGE}}$ (Ens.+TriviaQA)	**86.2**	**92.2**	**87.4**	**93.2**

> 질문에 대한 답을 문장에서 찾는 task

- 방식
- 각 토큰(T_i)에 대해
  → start / end 위치 예측
- 결과
- 기존 최고 성능 대비 큰 폭 향상
- 인간 성능 수준에 근접 또는 초과

3️⃣ SQuAD v2.0

System	Dev		Test	
	EM	F1	EM	F1
Top Leaderboard Systems (Dec 10th, 2018)				
Human	86.3	89.0	86.9	89.5
#1 Single - MIR-MRC (F-Net)	-	-	74.8	78.0
#2 Single - nlnet	-	-	74.2	77.1
Published				
unet (Ensemble)	-	-	71.4	74.9
SLQA+ (Single)	-		71.4	74.4
Ours				
$\text{BERT}_{\text{LARGE}}$ (Single)	78.7	81.9	80.0	83.1

> 답이 없는 경우까지 포함하는 QA task

- 특징
- answer 없음 가능

🔹 결과

- 기존 모델 대비 성능 향상
- 더 현실적인 QA 문제에서도 우수한 성능

4️⃣ SWAG

System	Dev	Test
ESIM+GloVe	51.9	52.7
ESIM+ELMo	59.1	59.2
OpenAI GPT	-	78.0
$\text{BERT}_{\text{BASE}}$	81.6	-
$\text{BERT}_{\text{LARGE}}$	**86.6**	**86.3**
Human (expert)†	-	85.0
Human (5 annotations)†	-	88.0

> 문장의 가장 자연스러운 다음 문장을 선택하는 task

🔹 방식

- 문장 A + 선택지 B 구성
- [CLS] 기반 classification

🔹 결과

- 기존 ELMo, GPT 대비 성능 향상

5️⃣ Ablation Study

	Dev Set				
Tasks	MNLI-m (Acc)	QNLI (Acc)	MRPC (Acc)	SST-2 (Acc)	SQuAD (F1)
$\text{BERT}_{\text{BASE}}$	84.4	88.4	86.7	92.7	88.5
No NSP	83.9	84.9	86.5	92.6	87.9
LTR & No NSP	82.1	84.3	77.5	92.1	77.8
+ BiLSTM	82.1	84.1	75.7	91.6	84.9

> BERT 구성 요소의 효과 분석

🔹 NSP 제거

- 성능 감소
  → 문장 관계 학습 중요

🔹 MLM 제거

- 성능 크게 감소

🔹 모델 크기 영향

- BERTLARGE > BERTBASE

👉 모델 크기 증가 → 성능 향상

5. 결론(배운 점)

💡

1️⃣ 핵심 기여
✅ 양방향(Bidirectional) Transformer 기반 pre-training 제안
✅ MLM + NSP를 통한 효과적인 학습 구조 설계
✅ 다양한 NLP task에 동일 구조 적용 가능

2️⃣ 주요 성과
✅ 여러 benchmark에서 SOTA 달성
➡️ 특히low-resource 환경에서도 높은 성능

3️⃣ 의의
✅ pre-training + fine-tuning 패러다임을 확립
✅ NLP 모델 설계 방식의 표준 제시

4️⃣ 결론

> 양방향 문맥을 활용한 사전 학습이 다양한 NLP task에서 높은 성능을 달성하는 핵심 요소임을 입증